Kubernetes　Chapter 12

Service全種とアーキテクチャを俯瞰する

4種のServiceとControl Plane/Worker Nodeの全体像を掴む

viserhaut — 2026

SECTION 01

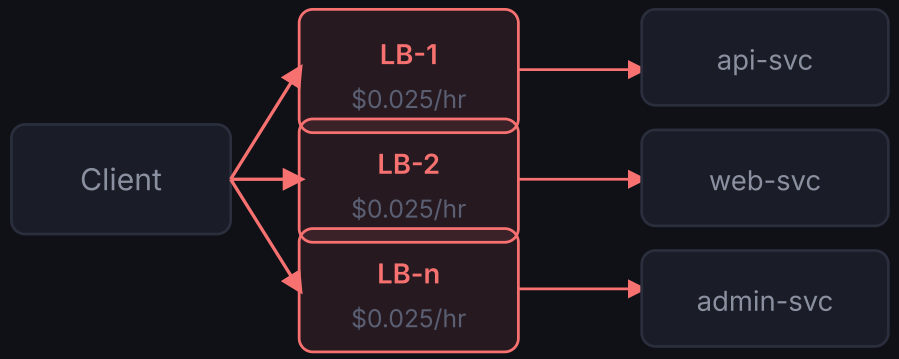
Service 4種を使い分ける

ClusterIP / Headless / NodePort / LoadBalancer — 4種比較

SERVICEタイプ	アクセス元	用途	AWSの相当
ClusterIP	クラスタ内のみ	マイクロサービス間通信	ALB Target Group（内部）
Headless	クラスタ内のみ	StatefulSet・DBのPod直接アクセス	Route53 Private（Pod IP返却）
NodePort	クラスタ外も可	ローカル開発・確認用	（非推奨）
LoadBalancer	クラスタ外（外部LB）	本番の外部公開（1サービス）	NLB（Network LB）

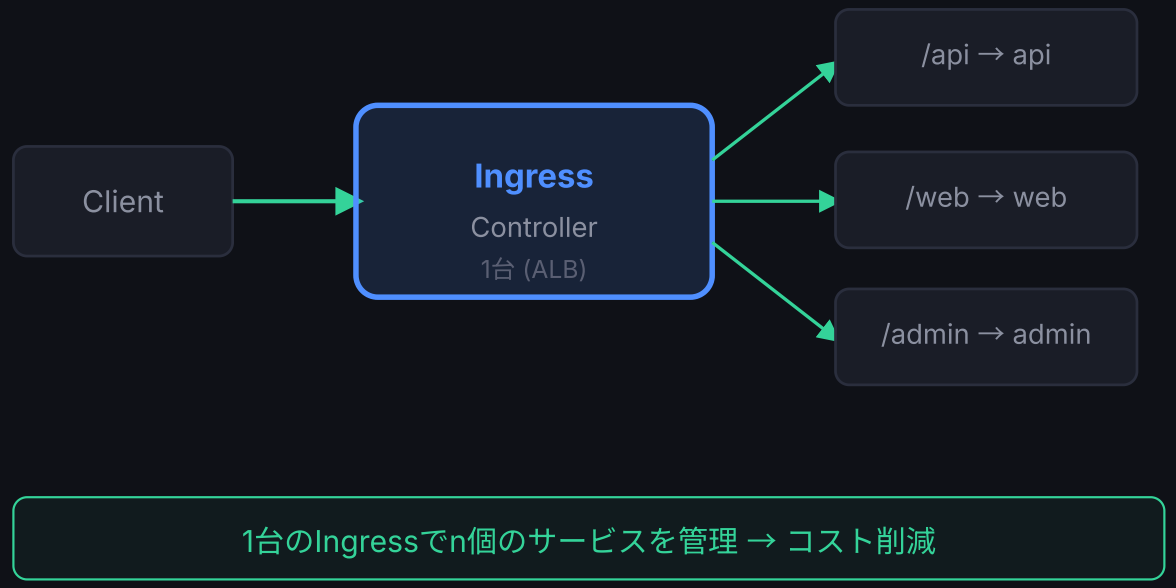
マイクロサービス増加でLoadBalancerは破綻する

- LoadBalancer Serviceは**サービスごとに外部LBが1台**必要。クラウドプロバイダーのLBリソースが必要
- マイクロサービスが10個あれば**LBが10台 → コストとIP枯渇が問題**になる
- また、各LBはL4（TCP）のみ対応。**パスベースのルーティングができない**



n個のLB × $0.025/hr → コスト爆発 + パスルーティング不可

IngressがLoadBalancerの代替になる理由

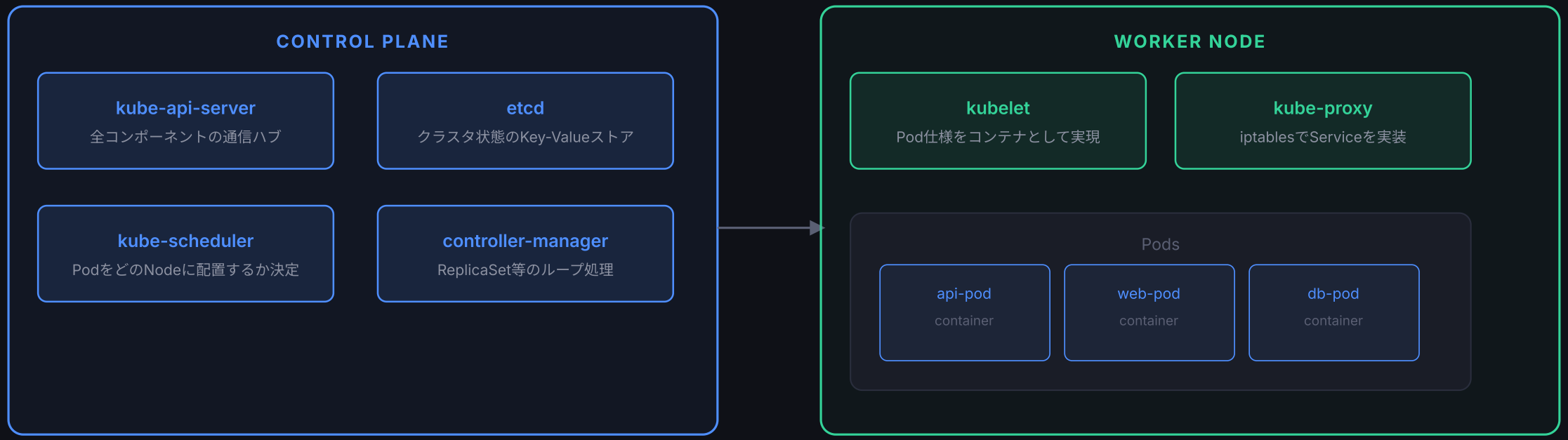


- **Ingress + Ingress Controller**で1台のエントリポイントから複数サービスへL7ルーティングができる
- AWSでは**ALB Ingress Controller**を使うとIngress定義からALBが自動プロビジョニングされる。1台のALBで複数サービスに対応
- TLS終端・ホスト名ルーティング・パスルーティングすべてを**1つのIngressマニフェストで管理**できる

SECTION 02

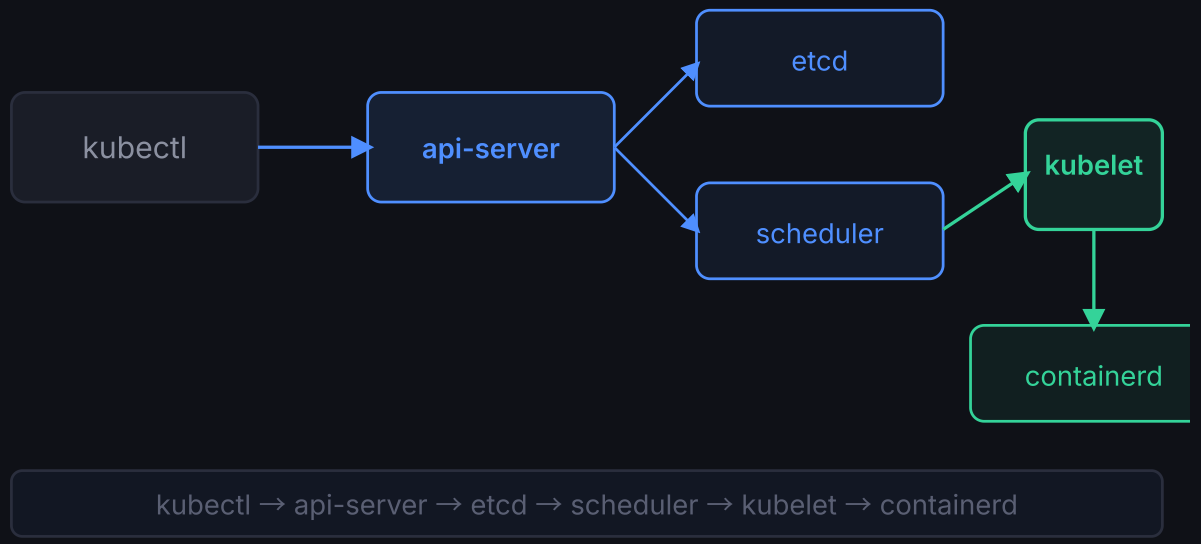
Kubernetes アーキテクチャ

Control Plane + Worker Node 全体構成図



kubeletがPod仕様を実現し、kube-proxyがネットワークを制御

- **kubelet**は各Worker Nodeで動くエージェント。kube-api-serverからPod仕様を受け取り、コンテナランタイム（containerd）にコンテナ起動を指示する
- **kube-proxy**はServiceが作られるとiptablesルールを書き換え、ClusterIPへのパケットをPod IPに転送する
- kubectl applyからPod起動まで: **kubectl → api-server → etcd → scheduler → kubelet → containerd** の順に処理される



コース振り返り — 3つの核心

01

宣言的APIがKubernetesの本質

「あるべき状態」をYAMLで宣言するだけで、Control Planeが自動的に現実状態を一致させ続ける

02

Service階層でネットワークを構成

ClusterIP→NodePort→LoadBalancer→Ingressの4層を状況に応じて使い分けることが設計の要点

03

AWSの知識がそのまま活きる

ECSサービス→Deployment、ALB→Ingress、EventBridge→CronJobと1対1で対応している。ECS経験者はすぐKubernetesに馴染める